

The Alberta Buck - Ethereum Implementation (DRAFT v0.6)

Perry Kundert

2026-04-20 12:00:00



This document presents a concrete Ethereum implementation of the Alberta Buck Architecture, specifying the five core smart contracts:

1. **BUCK_CREDIT** (ERC-721): An NFT representing an insurer's offer of parametric insurance on a real-world asset, with deterministic depreciation curves and piecemeal client activation.
2. **IdentityRegistry** (custom): An on-chain registry of per-address Pointcheval-Sanders signature credentials and ElGamal-encrypted identity points, verified via a BN254-precompile NIZK at registration time. This is the trust anchor every BUCK transfer is gated on – see the companion Identity and Proofs documents for the full cryptographic design.
3. **BUCK** (ERC-20): A fungible token minted against aggregated BUCK_CREDIT values, with on-chain credit-limit enforcement, default insurance premiums, and **identity-aware approve** and **transfer** flows that enforce the BUCK mutual-decryptability invariant on every counterparty pair via Chaum-Pedersen NIZK re-encryption proofs.
4. **Notes** (custom): A privacy-preserving denomination layer on top of BUCK. Clients escrow BUCK into a commitment pool by posting a Groth16 SNARK that ties the deposited **totalFace** to a batch of Poseidon-hash note commitments; later, any holder can redeem a single note by presenting a second Groth16 SNARK proving membership in the on-chain Tornado-style incremental Poseidon Merkle tree under a fresh nullifier. Redemption pays out via **BUCK.transferCarrying**, which weighted-merges the recipient's demurrage age with the pool's so the SNARK's **face == v = value-conservation** invariant holds at the ERC-20 boundary. See Notes and Proofs for the circuit design.

5. **BUCK_K** (custom): A controller that supplies the dynamic Value Stabilization Factor used in credit-limit computation. The first deployment uses `BuckKControllerStatic`, a governance-settable constant, so the identity and notes layers can land first; the eventual on-chain PID + commodity-basket oracle implementation (`BuckKController`) is fully specified below and slots in at the same interface.

This is the first of several planned implementation approaches. The ERC-721 approach is chosen for its clarity, wide tooling support, and natural fit for unique insured assets. Subsequent documents will examine ERC-1155 (semi-fungible batch credits), ERC-3643 (regulated security token with identity compliance), and ERC-4626 (vault-based credit pooling). The **Implementation Plan** at the end of this document specifies the concrete phased build order and tracks completion status.

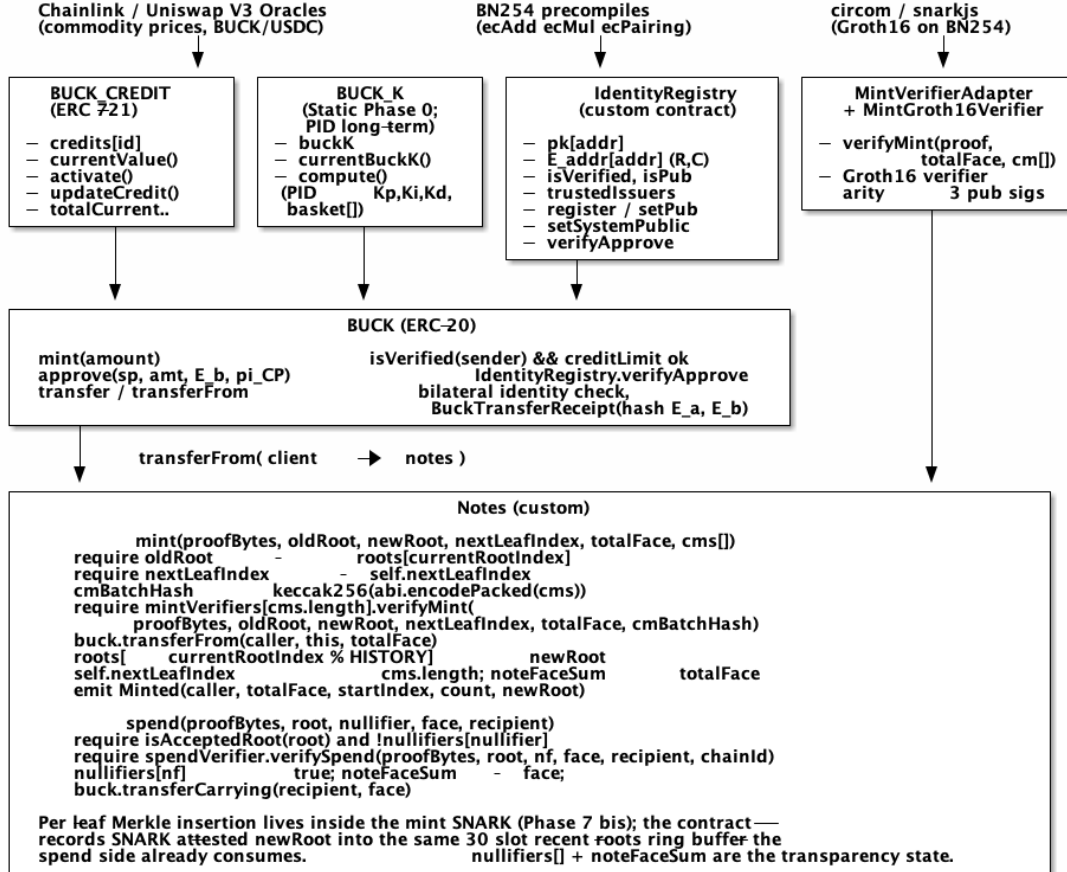
Contents

1	Contract Architecture Overview	5
2	ERC Standard Analysis for BUCK_CREDIT	6
2.1	Recommendation	6
3	BUCK_CREDIT: ERC-721 Insured Asset NFT	7
3.1	Data Model	7
3.2	Depreciation Models	7
3.2.1	Depreciation Examples	8
3.3	Activation (Execution) Model	8
3.4	Insurer Updates	8
4	IdentityRegistry: On-Chain Trust Anchor	8
4.1	Cryptographic Surface	9
4.2	Solidity Surface	9
4.3	Public-Claim Accounts	10
4.4	Issuer Trust	10
5	BUCK: ERC-20 Token	10
5.1	mint() Implementation	11
5.2	Identity-Aware approve()	11
5.3	Identity-Aware transfer() and transferFrom()	12
5.4	Demurrage and transferCarrying()	12
5.5	Default Insurance Premium	13
6	BUCK_K: Value Stabilization Controller	13
6.1	Process Variable and Setpoint	13
6.2	Solidity Implementation	14
6.3	PID Tuning Considerations	14
6.4	Adaptive Gain Scheduling (Future Enhancement)	15
7	Notes: Privacy-Preserving Denomination	15
7.1	Mint Circuit (circom, Groth16 on BN254)	15
7.2	Spend Circuit (circom, Groth16 on BN254)	16
7.3	Verifier Interfaces and Adapters	17
7.4	Notes Contract	17
8	Integration: Full mint() and transfer() Walkthrough	18
8.1	Step 0: Identity Registration (One-Time)	18
8.2	Step 1: Mint	19
8.3	Step 2: Approve Bob (Identity Handshake)	19
8.4	Step 3: Bob Pulls the Transfer	20
8.5	Step 4: Alice Denominates 125 BUCK as Notes	20
8.6	Step 5: Alice (or her recipient) Spends a Note	21
9	Gas Cost Analysis	21

10 Security Considerations	22
10.1 Oracle Manipulation	22
10.2 Credit Enumeration DoS	22
10.3 Insurer Trust	22
10.4 Reentrancy	22
11 Future Approaches	22
11.1 ERC-1155: Semi-Fungible Batch Credits	23
11.2 ERC-3643 (T-REX): Regulated Security Token	23
11.3 ERC-4626: Vault-Based Insurance Pool	23
11.4 L2 and Cross-Chain Deployment	23
12 Implementation Plan	23
12.1 Phase Completion Status (current)	24
12.2 Testing Strategy: Three Layers	24
12.3 Off-Chain Wallet (alberta_buck.wallet, Python)	25
12.4 Phase 0: Stub the Surrounding Contracts	25
12.5 Phase 1: BN254 Helper Library	25
12.6 Phase 2: IdentityRegistry Contract	26
12.7 Phase 3: Identity-Aware Buck ERC-20	26
12.8 Phase 4: Public-Claim Account Support	27
12.9 Phase 5: Notes Scaffolding	27
12.10Phase 6: Groth16 Mint Circuit + Verifier	28
12.11Phase 7: Spend Circuit + Nullifier Reveal	28
12.11.1 Resolved Items from the Notes Correctness Review	29
12.12Sequencing and Stop Conditions	30

1 Contract Architecture Overview

The Alberta Buck Ethereum implementation comprises five interacting contracts. The **IdentityRegistry** is the trust anchor of every BUCK transfer: **mint**, **approve** and **transfer** are all gated on the sender (and, where relevant, the recipient) holding a verified, registered identity credential. The **Notes** contract sits on top of BUCK as a public (**setSystemPublic**) counterparty whose commitments pool is extended only when the caller supplies a Groth16 SNARK binding the deposited **totalFace** to a batch of Poseidon note commitments.



The flow is:

1. An **insurer** creates a **BUCK_CREDIT** NFT representing their offer to insure a client's asset.
2. The **client** (asset owner) activates all or part of their credit, paying the current premium.
3. Before the client can interact with BUCK at all, they **register** a Pointcheval-Sanders identity credential and ElGamal-encrypted identity point with **IdentityRegistry**. The contract verifies the issuer's PS signature and the binding NIZK proof on-chain via the BN254 pairing precompile.
4. When the client calls **BUCK.mint()**, the contract requires **IdentityRegistry.isVerified(sender)**, then aggregates the client's active credits, applies depreciation, multiplies by the current **BUCK_K**, and mints BUCKs up to the resulting limit.

5. To send BUCKs, the sender first calls `BUCK.approve(spender, amount, E_bob, pi_CP)`, which routes the recipient’s re-encrypted identity ciphertext and Chaum-Pedersen proof through the `IdentityRegistry`; the proof binds `E_bob` to the sender’s registered credential without ever revealing the underlying identity point M .
6. `transfer` / `transferFrom` then perform a bilateral identity check (public-claim recipients skip the receipt requirement, private recipients must have a stored receipt fragment from a prior `approve`) and emit a `BuckTransferReceipt` event carrying only ciphertext hashes.
7. To denominate holdings as private notes, the client calls `Notes.mint(proofBytes, cm[], totalFace)`. The `MintVerifierAdapter` decodes the proof as `(uint256[2], uint256[2][2], uint256[2])` and forwards to the snarkjs-generated `MintGroth16Verifier`, which checks a Groth16 proof with public signals `[totalFace, cm[0], cm[1]]`. The circuit enforces `totalFace =  $\sum v_i$` and `cm_i = Poseidon(flavor_i, v_i, rho_i, idHash_i, predicate_i)` per note. The `Notes` contract is registered `setSystemPublic(true)` so its `transferFrom` pull from the minter satisfies the identity-aware BUCK transfer rules without itself holding a private credential.
8. BUCK_K ships in two stages. **Phase 0** uses `BuckKControllerStatic` with a governance-settable value, so the identity and notes layers can land first. The on-chain PID + commodity-oracle implementation (`BuckKController`, described in the BUCK_K section) is the eventual replacement.

2 ERC Standard Analysis for BUCK_CREDIT

Before diving into the ERC-721 implementation, we compare candidate standards against the BUCK_CREDIT requirements:

Requirement	ERC-721	ERC-1155	ERC-3643 (T-Rex)	ERC-721
Unique per-asset credit	Native	Possible (NFT mode)	Native	Poor
Insurer-mutable parameters	Custom role	Custom role	Built-in agent roles	Not d
Deterministic on-chain valuation	Custom view	Custom view	Custom view	share
Piecemeal client activation	Custom state	Fungible sub-amounts	Custom state	depos
Transfer restrictions	Custom	Custom	Built-in compliance	Open
Enumerable (for mint aggregation)	ERC721Enumerable	balanceOf per ID	Inherits ERC-20	Singl
Regulatory identity / KYC	Not built-in	Not built-in	ONCHAINID native	Not l
Tooling / ecosystem maturity	Excellent	Very good	Moderate (niche)	Exce

2.1 Recommendation

ERC-721 is the strongest first choice:

- Each BUCK_CREDIT is genuinely unique (specific asset, depreciation curve, insured value, premium schedule). This is precisely what ERC-721 models.
- The `ERC721Enumerable` extension provides `tokenOfOwnerByIndex()`, required for `BUCK.mint()` to aggregate all credits in an account.
- Custom extensions for dual-role access (owner activates, insurer updates) and deterministic valuation are straightforward.

- Widest tooling support, most audited implementations, lowest learning curve.

ERC-3643 is the strongest future candidate, especially if regulatory compliance (KYC, transfer restrictions, identity registry) becomes a deployment requirement. Its agent-role system maps naturally to the insurer/client relationship. However, its dependency chain (ONCHAINID, compliance modules, claim topics) adds significant complexity that is better deferred.

ERC-1155 is attractive if standardized credit "classes" emerge (e.g., all Calgary residential properties with the same depreciation model), allowing semi-fungible batching. For the initial system where each credit is individually appraised, the fungibility benefit doesn't apply.

ERC-4626 is a poor fit for individual credits but may be useful for the `InsurancePool` (where mutual insurance shares *are* fungible vault interests).

3 BUCK_CREDIT: ERC-721 Insured Asset NFT

3.1 Data Model

Each `BUCK_CREDIT` token is an `ERC721Enumerable` whose per-token `CreditParams` struct (`src/BuckCredit.sol`) carries all the insurer's current offer and the client's activation state. All monetary values are 18-decimal `BUCK`-equivalent units. Fields group naturally into three zones of mutability:

- **Immutable** at mint: `insurer` (the vendor address that may update this token), `assetClass`, `createdAt`.
- **Insurer-mutable** via `updateCredit`: `faceValue`, `depreciationFloor`, `depType`, `depRate`, `depStartAt`, `premiumRate`, `lastUpdated`.
- **Client-mutable** via `activate`: `activatedValue` ($\leq$ `faceValue`), `lastActivatedAt`.

`DepreciationType` is one of `NONE` (land, gold), `LINEAR` (constant annual reduction), or `DECLINING_BALANCE` (percentage of remaining value). Three events – `CreditCreated`, `CreditUpdated`, `CreditActivated` – record every state change. The per-token `insurer` field lets a single deployment host many insurers; `assetClass` cannot be reclassified after mint.

3.2 Depreciation Models

Depreciation is computed deterministically from the NFT's on-chain parameters in `currentValue(tokenId)` – a pure view with no oracle input, gas-free to query. The model branches on `depType`:

- **NONE**: `currentValue` is the activated face value, unchanged.
- **LINEAR**: loss accrues at `depRate` basis points per year on the depreciable portion (`faceValue` - `floor`), clamped at the `depreciationFloor`.
- **DECLINING_BALANCE**: continuous-compounded version using an in-contract `_expNeg` (6th-order Taylor series, accurate to $<0.01\%$ for $x < 3.0$ – i.e. up to $\sim 300\%$ cumulative depreciation). A production deployment should swap in `PRBMath.exp()` or `ABDKMath64x64` for wider-range precision.

In every case the activated portion depreciates proportionally: `currentValue = depreciatedFace * activatedValue / faceValue`.

3.2.1 Depreciation Examples

Asset	Type	Rate	Floor	30yr value (of \$100K)
Land	NONE	–	–	\$100,000
House (bldg)	LINEAR	200bp	20%	\$52,000
Vehicle	DECLINING_BALANCE	1500bp	5%	\$5,633
Farm equip	DECLINING_BALANCE	1000bp	10%	\$16,791
Gold in vault	NONE	–	–	\$100,000

A residential property would typically have *two* BUCK_CREDITS: one for the land (NONE, full face value indefinitely) and one for the structure (LINEAR or DECLINING_BALANCE, with a floor representing salvage/lot-clearing value).

3.3 Activation (Execution) Model

The client executes (activates) their credit piecemeal through `activate(tokenId, amount)` – gated on `ownerOf(tokenId) = msg.sender` – and capped at the token’s `faceValue`. Activation increases the `activatedValue` and stamps `lastActivatedAt`; the *premium* for the newly activated amount is collected by `BUCK.mint()`, not here.

`totalCurrentValue(account)` walks `ERC721Enumerable.tokenOfOwnerByIndex` and sums `currentValue(tokenId)` across all tokens owned by `account` – this is the single call `BUCK.mint()` makes to compute the credit limit.

The activation model separates the *decision to use credit* from the *act of minting BUCKs*. A client can activate \$200K of their \$500K home credit today, and months later activate \$100K more – possibly at a different premium rate if the insurer has updated the NFT in the interim.

3.4 Insurer Updates

The insurer can reappraise the asset, change depreciation parameters, and adjust premium schedules. These changes affect *future* credit limit computations, including already-activated amounts (since the depreciation function is computed from current parameters, not historical ones).

`updateCredit(tokenId, newFaceValue, newDepreciationFloor, newDepType, newDepRate, newDepStartAt, newPremiumRate)` is gated on `msg.sender = credits[tokenId].insurer`. If `newFaceValue < activatedValue` the contract caps `activatedValue` downward in the same call; `assetClass` and `insurer` are never changed.

Note that if the insurer reduces `faceValue` below the current `activatedValue`, the activated amount is capped downward. This could push the client’s outstanding BUCK balance above their credit limit, triggering the default insurance mechanism described below.

4 IdentityRegistry: On-Chain Trust Anchor

The `IdentityRegistry` is the binding between Ethereum addresses and the privacy-preserving identity machinery described in the companion documents Identity with Anonymity, Identity Example – Data Flow, and Cryptographic Proofs. Every BUCK `mint`, `approve` and `transfer` consults this registry; an unregistered address cannot hold or move BUCKs.

The registry stores three pieces of state per address:

Field	Type	Source
<code>pk[a]</code>	G_1 point	Identity public key generated by the wallet
<code>E_addr[a]</code>	(R, C) in G_1^2	ElGamal encryption of $M = m \cdot G$ under <code>pk</code>
<code>isVerified</code>	bool	True after PS + NIZK proofs verify on-chain
<code>isPublic</code>	bool	Opt-in: account publishes M (DeFi pools, etc.)

It also holds a small set of trusted issuer public keys `trustedIssuers[issuerAddr] = (X, Y)` in G_2^2 , populated by governance. These are the only PS public keys whose signatures the registry will accept at `register()` time.

4.1 Cryptographic Surface

The registry implements three on-chain verifications, all reducible to BN254 precompile calls:

1. **PS signature verification at registration:** $e(\text{sigma}'_1, X + m \cdot Y) = e(\text{sigma}'_2, g_2)$, where the verifier reconstructs the relation linearly in `m` via the binding NIZK in step 2. Costs $\sim 147K$ gas (3-4 `ecPairing` pairs).
2. **NIZK proof binding the rerandomized PS signature to the ElGamal ciphertext.** A Schnorr-family sigma protocol with three response scalars and a Fiat-Shamir challenge bound to `(sigma', E_new, pk_new, registrant_address)`. Costs $\sim 30K$ gas of `ecMul/=ecAdd` on top of the pairing check. Total registration cost is $\sim 235K$ gas (one-time).
3. **Chaum-Pedersen proof of re-encryption correctness at approve time.** The recipient identity ciphertext `E_bob` and proof `pi_CP = (e, s_1, s_2)` arrive in `approve()` calldata; the registry reads `E_alice` from storage (anchoring the proof to the registered credential) and verifies via 3-4 `ecMul` + 3-4 `ecAdd` operations and a Fiat-Shamir reconstruction. Costs $\sim 29K$ gas per counterparty pair.

4.2 Solidity Surface

`src/IdentityRegistry.sol` uses the BN254 helper library for all curve math. The shape of the interface is what matters for the rest of this document:

```

struct G1Point { uint256 X; uint256 Y; }
struct G2Point { uint256[2] X; uint256[2] Y; }

struct ElGamalCT { G1Point R; G1Point C; } // (R, C) = (r*G, r*pk + M)
struct PSSig { G1Point s1; G1Point s2; } // rerandomized PS signature
struct PSPubKey { G2Point X; G2Point Y; } // issuer (X, Y) in G_2

struct RegistrationProof { uint256 e, s_m, s_r; G1Point A_ps, T_C, T_R; }
struct CPProof { uint256 e, s1, s2; G1Point T1, T2, T3; }

function register(address issuer, G1Point pk, ElGamalCT E, PSSig sigma, RegistrationProof proof)
function setPublic(bool isPublic);
function setSystemPublic(address account, bool isPublic); // governance
function trustIssuer(address issuer, PSPubKey ipk); // governance
function verifyApprove(address sender, address spender,
                      ElGamalCT E_bob, CPProof pi) view returns (bool);

```

Design notes on the surface:

- Per-address state is `pk`, `E_addr` ((R,C) ciphertext of the identity point `M`), `isVerified`, `isPublic`, and `issuerOf` – all public-readable mappings.
- `register` runs three checks in order: (1) recompute the Fiat-Shamir challenge with `msg.sender` binding, (2) verify the Schnorr-family NIZK in `G_1`, (3) verify the PS pairing relation in `G_2`. An `already registered / untrusted issuer` guard precedes them. Total cost $\sim 235K$ gas (one-time per address).
- `setPublic` and `setSystemPublic` are the only two mutators after registration. The system variant is governance-gated and exists for contracts (Notes, InsurancePool) that have no private key to sign CP proofs with – Notes is registered public this way by `script/Deploy.s.sol`.
- `verifyApprove` reads `E_alice` from storage so the caller cannot substitute a fake credential; the `CPProof` structure carries `T1/T2/T3` because the transcript binds `T1` and `T2` separately (only `T3` is recoverable from the verification equations).
- The `RegistrationProof` carries the PS-side commitment $A_{ps} = m_{\tilde{}} * \sigma'_1$ alongside the two ElGamal-side commitments `T_C`, `T_R` – A_{ps} is indispensable for the pairing-product check and cannot be reconstructed from the other responses.
- Governance hooks round out the surface: `revokeIssuer`, `transferGovernance`. Revocation does not un-register existing accounts; they were verified once at registration and are never re-verified.

4.3 Public-Claim Accounts

DeFi infrastructure – AMM pools, vaults, oracles – cannot generate Chaum-Pedersen proofs at `approve` time, because the contract has no private key. These accounts call `setPublic(true)` after `register()`; from then on, counterparties of a public account skip the proof requirement and the on-chain receipt is constructed directly from the published identity point. Public accounts forfeit content privacy: their identity `M` is observable on-chain, but they retain unforgeability and the audit trail.

4.4 Issuer Trust

`trustedIssuers` is the only governance-controlled trust anchor in the system. Adding a new issuer admits a new origin of registrable identities; removing one stops new registrations under that issuer (existing registrations are unaffected – they were verified at registration time and are never re-verified). Epoch-based credential renewal, key revocation, and issuer rotation are described in the Identity: Epoch-Based Credential Renewal document.

5 BUCK: ERC-20 Token

The BUCK token is an **identity-aware** ERC-20. Beyond the usual `mint` / `transfer` / `approve` surface, every state-changing call routes through the `IdentityRegistry`:

- `mint()` refuses to issue BUCKs to an unverified address.

- `approve()` takes the recipient’s re-encrypted identity ciphertext and a Chaum-Pedersen NIZK proof, and stores a *receipt fragment* binding the (sender, spender) pair to that ciphertext.
- `transfer()` and `transferFrom()` enforce a **bilateral** identity check using the receipt fragments and the public-claim flag, then emit a `BuckTransferReceipt` carrying only ciphertext hashes – no plaintext identity ever touches the chain.

This is the on-chain enforcement of the mutual-decryptability invariant: every transfer leaves behind a permanent record from which both counterparties (and only those counterparties, by decrypting their respective ciphertext) can reconstruct the identity of the other.

5.1 mint() Implementation

`src/Buck.sol` is constructed with `(buckCredit, buckK, identity, insurancePool).mint(amount)` performs the seven-step sequence:

1. Require `identity.isVerified(msg.sender)`.
2. Read `creditValue = buckCredit.totalCurrentValue(msg.sender)`.
3. Read `k = buckK.currentBuckK()` (Phase-0 static getter or eventual PID `compute()`).
4. `maxLimit = creditValue * k / 1e18`; the stored per-account `storedLimit` ratchets upward only.
5. Refuse if `balanceOf(sender) + amount > storedLimit`.
6. Compute the default-insurance premium (see below).
7. `_mint(sender, amount - premium), _mint(insurancePool, premium), emit Minted(account, amount, premium, creditValue, buckKValue, newLimit)`.

A symmetric `burn(amount)` just calls `_burn(msg.sender, amount)` for repayment.

5.2 Identity-Aware approve()

The shipped signature is

```
approve(address spender, uint256 amount, IdentityRegistry.ElGamalCT E_bob, IdentityRegistry.Cpi_CP).
```

The parameterless ERC-20 `approve(address, uint256)` reverts with `BUCK: use identity-bound approve` – the identity-bound form is mandatory. Both `sender` and `spender` must be verified (`spender` may instead be `isPublic`). For private spenders the call routes through `identity.verifyApprove(...)`; on success the receipt fragment `keccak256(abi.encode(E_bob.R.X, E_bob.R.Y, E_bob.C.X, E_bob.C.Y))` is stored at `_receiptFragments[sender][spender]`. For `isPublic` spenders the CP proof is skipped and the fragment is set to a hash of the spender’s published M via `_publicReceiptHash(spender)`. The ERC-20 allowance is then written via `_approve(sender, spender, amount)`.

The `approve` call costs roughly 29K (CP verify) + 46K (standard ERC-20 approve) + 22K (`receipt SSTORE`) = ~97K gas, paid once per (sender, spender) pair. Subsequent transfers between the same counterparties incur only the bilateral check and the receipt event (~5K gas of additional work above a baseline ERC-20 transfer).

5.3 Identity-Aware transfer() and transferFrom()

Both `transfer(to, amount)` and `transferFrom(from, to, amount)` route through `_identityCheckedTransfer`, which enforces:

- `isVerified(from)` and `(isVerified(to) or isPublic(to))`.
- For private `to`, `_receiptFragments[from][to] != 0` (the prior CP-bound approve must exist).
- The `to` side of the receipt event is the stored fragment hash, or `_publicReceiptHash(to)` if the recipient is public.
- The `from` side is the mirrored fragment `_receiptFragments[to][from]` if a reciprocal approve exists, or `_publicReceiptHash(from)` for public senders, or `bytes32(0)` for a one-shot transfer with no prior reciprocal approve. The receipt still records the asymmetric attestation – auditors reading the event know which side was CP-bound.

After the underlying `_transfer` settles, `BuckTransferReceipt(from, to, amount, fromHash, toHash)` is emitted. `_publicReceiptHash(account)` is `keccak256(abi.encode(R.X, R.Y, C.X, C.Y, "PUBLIC"))` over the account's stored ElGamal ciphertext.

The `BuckTransferReceipt` event is the auditable trail: `fromCipherHash` and `toCipherHash` are `keccak256` commitments over the corresponding ElGamal ciphertexts. Counterparties can match a receipt to their off-chain copies of E_{alice}, E_{bob} by recomputing the hash; observers without the underlying ciphertexts learn nothing beyond the fact of a transfer.

5.4 Demurrage and transferCarrying()

BUCK carries a continuous on-chain demurrage accrual to drive a perpetual Jubilee schedule. All balance accounting is in *raw* units; the displayed `balanceOf(a)` subtracts a per-account `feeOwing(a) = raw_balance(a) * (cumIndexNow - _indexAtLastTouch[a]) / SCALE`. `cumIndex` grows linearly at `BASE_RATE_PER_SEC * (now - genesis)`. Two transfer flavours coexist:

- **Deducting** (default ERC-20 path): the sender's accumulated fee is burned to the Jubilee recipient's index slot; sender's `_indexAtLastTouch` resets to `cumIndex`; the recipient's index is weighted-merged toward `cumIndex` so its prior fee debt is preserved.
- **Carrying** (`transferCarrying(to, amount)`): no fee burn. `amount` raw units move atomically with the sender's accrued demurrage age – the recipient's index becomes the amount-weighted average of its old index and the sender's `_indexAtLastTouch`: `_indexAtLastTouch[to] = (br * _indexAtLastTouch[to] + amount * idxFrom) / (br + amount)`. The sender's index is unchanged; sender keeps the residual age on its remaining balance.

`transferCarrying` is the entry point `Notes.spend` calls when paying out a redeemed face value: it is the reason a recipient absorbs the pool's average demurrage age (the anonymity cost – and upside, for anyone who held a note longer than the pool average) rather than a discontinuous spike of settled fee. The Jubilee accumulates fee burns into a single sink whose own `_indexAtLastTouch` is itself weighted-merged so disbursements remain age-correct.

5.5 Default Insurance Premium

The premium scales with utilization: the closer the account's outstanding BUCKs approach the credit limit, the more expensive each additional BUCK becomes. This mirrors the architecture document's description of Parametric Default Insurance.

The shipped curve is quadratic in utilization: `premium_rate = BASE_RATE + utilization^2 * SCALE_RATE`, with `BASE_RATE = 50bp` (0.50%) and `SCALE_RATE = 450bp` (4.50% additional at 100% utilization). Concretely: ~0.5% at 0% utilization, ~1.75% at 50%, ~4.55% at 90%; 100% is unreachable because the limit check itself rejects. The `_computePremium(account, mintAmount, limit)` helper computes utilization on the post-mint balance and returns the BUCK amount due to the insurance pool.

The quadratic scaling creates a natural disincentive against over-leveraging. A client using 50% of their credit pays roughly 1.75% of each minted BUCK to the insurance pool; at 90% they pay 4.55%. The insurance pool accumulates these premiums to cover defaults (accounts where asset depreciation or `BUCK_K` changes push outstanding BUCKs above the credit limit).

6 BUCK_K: Value Stabilization Controller

The `BUCK_K` contract maintains purchasing-power parity between the BUCK and its commodity basket. It ships in two stages behind a shared one-function interface (`currentBuckK()` `external view returns (uint256)`) so the Buck contract is invariant to which stage is deployed:

- **Phase 0** – `BuckKControllerStatic`: a governance-settable constant. Constructor (`uint256 _buckK, address _governance`); functions `currentBuckK()`, `compute()` (no-op, returns the stored value), `setBuckK(uint256)` (governance-only), `transferGovernance(address)`. This is the version currently deployed alongside the identity and notes layers so those layers can land first.
- **Phase Later** – `BuckKController`: a full PID implementation, identical in structure to the project's ownercredit PID controller, ported to Solidity fixed-point arithmetic. It reads a commodity basket from Chainlink feeds and the BUCK/USDC price from a Uniswap V3 TWAP, runs a P/I/D update on `compute()` once per `dT` seconds, and writes the new `buckK`.

The PID implementation is documented in full below; the Buck ERC-20 reads only `currentBuckK()` from either variant, so swapping one for the other is a constructor argument.

6.1 Process Variable and Setpoint

PID Term	Source	Meaning
Process Variable	BUCK/USDC Uniswap V3 TWAP	What a BUCK actually trades for
Setpoint	Commodity basket oracle sum	What a BUCK <i>should</i> be worth (basket cost)
Error	Setpoint - Process	Positive = BUCK undervalued, negative = overvalued
Output (<code>BUCK_K</code>)	PID computation	Credit limit multiplier (1.0 = neutral)

When BUCK trades *below* basket value (positive error), `BUCK_K` increases, expanding everyone's credit limit. This floods the market with cheap BUCKs, which arbitrageurs buy to acquire underpriced commodities, driving the BUCK price back up. The converse reduces `BUCK_K` when BUCKs are overvalued.

6.2 Solidity Implementation

`src/BuckKController.sol` is a straight Solidity port of the JavaScript PID controller in owner-credit, with 18-decimal fixed-point arithmetic and BN254-friendly types. The shipped shape is:

- **Configuration:** `int256 Kp, Ki, Kd` (gains), `uint256 dT` (minimum update interval), `uint256 buckKMin, buckKMax` (output clamps), `buckUsdcPool, twapInterval, governance`.
- **State:** `int256 P, I, D, uint256 lastUpdate, uint256 buckK` (output, `1e18` = neutral). `basket[]` is an array of `BasketComponent { AggregatorV3Interface feed; uint256 weight; uint8 feedDecimals; }`.
- **Entry point:** `compute()` returns (`uint256`). If `now - lastUpdate < dT` it returns the cached `buckK` without state change. Otherwise it reads `basketCost` (Chainlink-weighted set-point), `buckPrice` (Uniswap V3 TWAP, process variable), runs one PID cycle $= \text{rawOutput} = 1e18 + \text{newP} * Kp$
 $- \text{newI} * Ki + \text{newD} * Kd$ (each term divided by `1e18`), clamps to `[buckKMin, buckKMax]` with
anti-windup on the integral term, writes the new state, and emits `BuckKUpdated`.
- **View:** `currentBuckK()` view returns (`uint256`) – the Buck contract calls this for non-mutating reads.
- **Governance:** `setGains, setDT, addBasketComponent`.

The TWAP price helper `_getBuckPrice()` uses Uniswap’s `OracleLibrary.consult(pool, twapInterval)` to convert a `tickCumulatives` delta into an 18-decimal price; omitted here because it is mechanical Uniswap V3 boilerplate. The basket helper sums `price * weight` with per-feed decimal normalisation.

6.3 PID Tuning Considerations

The PID gains determine how aggressively `BUCK_K` responds to price deviations:

Gain	Effect of Increasing	Risk of Over-tuning
Kp	Faster response to price deviations	Oscillation, over-correction
Ki	Eliminates steady-state error (persistent mispricing)	Integral windup, slow recovery
Kd	Dampens oscillation, anticipates trends	Noise amplification

Initial gains should be conservative (high `Kd` relative to `Kp`, low `Ki`) to favor stability over responsiveness. The `dT` parameter (minimum update interval) acts as an additional dampening mechanism – a 1-hour `dT` means `BUCK_K` changes at most once per hour regardless of how many mints occur, preventing high-frequency oscillation.

The TWAP interval (`twapInterval`) provides manipulation resistance: a 30-minute TWAP means an attacker would need to sustain a price manipulation for 30 minutes to meaningfully affect `BUCK_K` – a prohibitively expensive proposition on a liquid pool.

6.4 Adaptive Gain Scheduling (Future Enhancement)

The current implementation uses governance-set fixed gains. A future enhancement could implement on-chain gain scheduling where K_p , K_i , and K_d are adjusted based on observed error dynamics:

- **Low volatility regime** (error variance below threshold): reduce K_p , increase K_i for tighter long-term tracking.
- **High volatility regime** (error variance above threshold): increase K_p and K_d for faster response, reduce K_i to prevent windup.
- **Regime detection**: maintain a rolling variance of the error signal over a window of N updates.

This maps directly to the `PIDController.reset()` pattern in the JavaScript implementation, where gains can be hot-swapped without spiking the derivative term.

7 Notes: Privacy-Preserving Denomination

BUCK transfers carry a public (`from`, `to`, `amount`) quartet. For private value movement, holders denominate their BUCKs as **notes**: a pair of Poseidon-hash commitments posted to the **Notes** contract, backed by an equal amount of BUCK locked in that contract. Notes do not leak values, flavours, or identity bindings – the public state is just a list of opaque 32-byte commitments and a running face-value total.

The design is specified in full in Notes and the underlying SNARK construction in Proofs. This section documents the Ethereum surface: the **Notes** contract, the **IMintVerifier** interface, the circom Groth16 mint circuit, and the adapter that bridges the two.

7.1 Mint Circuit (circom, Groth16 on BN254)

Phase-7-bis ships `circuits/mint_batch.circom`, a parameterised batch-mint circuit that subsumes the per-leaf on-chain Poseidon insertion of Phase 7. One compile per pinned N (recommended $\{16, 128, 1024\}$); each pin produces its own `MintGroth16Verifier_N*.sol` and registers in `mintVerifiers[N]` on **Notes**. The public-signals arity is 5 regardless of N .

Signal	Role	Description
<code>oldRoot</code>	public	Live note Merkle root the prover read from chain
<code>newRoot</code>	public	Root after folding <code>cm[0..N-1]</code> starting at <code>nextLeafIndex</code>
<code>nextLeafIndex</code>	public	Live tree size the prover read from chain
<code>totalFace</code>	public	Sum of note values, range-bounded to 128 bits
<code>cmBatchHash</code>	public	<code>keccak256(abi.encodePacked(cm[0..N-1]))</code> – ties calldata to the SNARK
<code>flavor[i]</code>	private	Note flavour (A1/A2/B1 from Notes design)
<code>v[i]</code>	private	Note value (18-decimal integer, range-bounded to 128 bits)
<code>rho[i]</code>	private	Per-note randomness
<code>idHash[i]</code>	private	Hash of the identity binding for this note
<code>predicate[i]</code>	private	Predicate commitment (zero for the plain mint path)
<code>pathSiblings[i][d]</code>	private	Depth-20 sibling hashes used to fold leaf <code>i</code> into the rolling tree

Constraints: N Poseidon-5 opening checks, Num2Bits(128) range bounds on each `v[i]` and on `totalFace` (closing the field-overflow attack), one sum check (`totalFace == sum v[i]`), N x TREE_DEPTH Poseidon-2 hashes implementing the rolling filled-subtrees insertion from `oldRoot` to `newRoot`, and one keccak-style hash check binding `cmBatchHash` to the per-leaf commitments. Total constraints work out to ~6K R1CS per leaf (~1.7K opening + ~4.3K insertion); see `alberta-buck-notes-rollup` for per-N R1CS / PTAU sizing. `scripts/snark/setup.sh` now runs one phase-2 setup per pinned N, exporting `src/MintGroth16Verifier_N*.sol` for each. `src/SpendGroth16Verifier.sol` is unchanged from Phase 7.

```
interface IMintGroth16 {
    function verifyProof(
        uint256[2]      calldata a,
        uint256[2][2]   calldata b,
        uint256[2]      calldata c,
        uint256[3]      calldata pubSignals    // [totalFace, cm[0], cm[1]]
    ) external view returns (bool);
}
```

The trusted-setup contributions in `setup.sh` use a fixed dev-only entropy string. A production deployment must run a real multi-party ceremony; the same script is the scaffolding.

7.2 Spend Circuit (circom, Groth16 on BN254)

`circuits/spend.circom` proves the redemption of a single B-shape note under an on-chain Merkle root. Public inputs are `[noteRoot, nullifier, face, recipient, chainId]`; private witness is `(flavor, v, rho, idHash, predicate, pathElements[20], pathIndices[20])`. The circuit enforces five constraint groups:

- **(R)** Num2Bits(128) on `face` and on `v`.
- **(C)** `cm == Poseidon(5)(flavor, v, rho, idHash, predicate) =`.
- **(M)** Tornado-style Merkle inclusion of `cm` under `noteRoot` at path `(pathElements, pathIndices)`: a chain of Poseidon(2) hashes with `Switcher` selectors ordering the left/right inputs at each level.
- **(N)** B-shape nullifier `nullifier == Poseidon(3)(rho, idHash, 4242) =` – the constant tag 4242 domain-separates the B-path; the A-path will use its own tag when `spend_a.circom` ships.
- **(B)** Ghost bind `face * 1 == v * 1 =` and a quadratic anchor `x*x` guarantees the prover committed to the value actually being redeemed (not an arbitrary smaller face).

`snarkjs` compiles the circuit to roughly 12,000 R1CS constraints; the 2^{15} pot is shared with `mint`. The on-chain `SpendGroth16Verifier` accepts a 5-element `pubSignals` tuple; the `SpendVerifierAdapter` presents the narrower `ISpendVerifier.verifySpend(proof, root, nullifier, face, recipient, chainId)` to the `Notes` contract.

7.3 Verifier Interfaces and Adapters

Notes is decoupled from the specific SNARK toolchain via two narrow interfaces:

```
interface IMintVerifier {
    function verifyMint(bytes proof, uint256 totalFace,
        uint256[] commitments, address issuer) external view returns (bool);
}

interface ISpendVerifier {
    function verifySpend(bytes proof, uint256 root, uint256 nullifier,
        uint256 face, address recipient, uint256 chainId)
        external view returns (bool);
}
```

Two implementations of each interface ship:

- **Stubs** (StubMintVerifier, StubSpendVerifier): governance-controlled booleans. Used in deployment bootstraps and unit tests that exercise state transitions unrelated to the SNARK itself.
- **Adapters** (MintVerifierAdapter, SpendVerifierAdapter): decode proof as `abi.encode(uint256[2], uint256[2][2], uint256[2])`, build the public-signals tuple (5 entries for mint – [oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash] – and 5 entries for spend), and forward to the snarkjs-generated `MintGroth16Verifier_N*` / `SpendGroth16Verifier`. The mint adapter is per-N: `mintVerifiers` is a mapping(`uint256 => IMintVerifier`)= keyed on `cms.length`, with one verifier registered per pinned batch size (recommended {16, 128, 1024}).

Swapping either verifier is a single governance call (`Notes.setMintVerifier(N, address)` / `Notes.setSpendVerifier(address)`); the Notes contract’s own storage and event surface are invariant.

7.4 Notes Contract

`src/Notes.sol` is constructed with (`buck`, `mintVerifierRegistry`, `spendVerifier`, `governance`). Its state is intentionally lean – the per-leaf Poseidon insertion math that Phase 7 had on chain now lives inside the mint SNARK, and the contract just records the SNARK-attested `newRoot`:

- **Verifier handles:** per-N `mintVerifiers[N]` (governance-swappable per pinned N), `ISpendVerifier spendVerifier` (governance-swappable singleton). No on-chain Poseidon precompile is required for mint – it remains relevant for legacy spend- side off-chain tree reconstruction tools, but the contract no longer calls it.
- **Audit and double-spend state:** mapping `nullifiers`, `uint256 noteFaceSum`.
- **Tree state:** `uint32 nextLeafIndex` and `uint256[ROOT_HISTORY_SIZE=30] roots` with `uint8 currentRootIndex` selecting the live root. `filledSubtrees`, `zeros[]`, and `commitmentExists` from Phase 7 are retired – the rolling tree state lives in the wallets’ local mirrors and inside the SNARK; duplicate commitments are economically self-punishing via the deterministic nullifier `Poseidon3(rho, idHash, 4242)` (one spend nullifies every copy of the same opening; Alice loses any BUCK she paid to mint duplicates).

`mint(proof, oldRoot, newRoot, nextLeafIndex, totalFace, cms)` asserts `cms.length > 0`, checks `oldRoot = roots[currentRootIndex]=` and `nextLeafIndex = self.nextLeafIndex=` (stale-state guards), recomputes `cmBatchHash = keccak256(abi.encodePacked(cms))`, dispatches to `mintVerifiers[cms.length]`, calls `mintVerifiers[cms.length].verifyMint(oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash)`, pulls `totalFace` BUCK via `transferFrom(caller, this, totalFace)`, writes `newRoot` into the ring buffer at `(currentRootIndex + 1) % ROOT_HISTORY_SIZE`, advances `nextLeafIndex` by `cms.length`, bumps `noteFaceSum` by `totalFace`, and emits a single `Minted(issuer, totalFace, startIndex, count, newRoot)` event. Per-leaf `Appended` events from Phase 7 are gone: the commitments themselves live in the transaction calldata for off-chain tree replayers and indexers to read.

`spend(proof, root, nullifier, face, recipient)` is unchanged from Phase 7: requires `recipient != 0`, `face > 0`, that `root` is still in the recent-roots window (`isAcceptedRoot(root)`), and that `nullifier` has not been seen. It forwards `(proof, root, nullifier, face, recipient, block.chainid)` to `spendVerifier.verifySpend`, marks the nullifier burned and decrements `noteFaceSum` before calling `IBuckCarrying(address(buck)).transferCarrying(recipient, face)` – the carrying path moves the pool’s average demurrage age to the recipient atomically with the face value. A `Spent(nullifier, face, recipient)` event closes the spend.

Phase 7-bis closes the same loop as Phase 7 but at *N-independent on-chain cost*: credit-backed BUCKs in via `mint` (~380K + ~500 gas/leaf calldata) -> opaque commitments fold into the rolling root via the SNARK -> `spend` redeems any committed commitment against any recent root with a fresh nullifier, paying the recipient via `transferCarrying` so the SNARK’s `face == v=` invariant is preserved at the ERC-20 boundary. The pivot is documented in alberta-buck-notes-rollup-mint.org.

8 Integration: Full mint() and transfer() Walkthrough

Consider Alice, who owns a \$500K Calgary home (land: \$200K, structure: \$300K). Two insurers (LandCo, HomeCo) have issued BUCK_CREDITS:

Token	Insurer	Face Value	Depreciation	Activated	Current Value
#42	LandCo	200,000	NONE	200,000	200,000
#43	HomeCo	300,000	LINEAR, 200bp, 20% fl	150,000	141,000
Total					341,000

The structure has depreciated 6% over 3 years (LINEAR at 2%/yr applied to the 80% depreciable portion). Alice activated all of the land credit but only half of the structure credit.

8.1 Step 0: Identity Registration (One-Time)

Before Alice can interact with BUCK at all, she must hold a verified identity credential:

1. **KYC ceremony** (off-chain): a trusted issuer (the Alberta government, ATB Financial, etc.) verifies Alice’s documents and signs her identity scalar $m = H(\text{identity_data})$ with their PS key, producing $\sigma = (\sigma_1, \sigma_2)$. Alice stores (m, σ) in her wallet.
2. **Derivation** (off-chain): Alice’s wallet rerandomizes σ into σ' , generates a fresh identity key pair $(sk_{\text{alice}}, pk_{\text{alice}})$, encrypts her identity point $M = m \cdot G$ under pk_{alice} , and produces a NIZK proof π binding σ' to that ciphertext.

3. **On-chain registration:** Alice calls `IdentityRegistry.register(issuer, pk_alice, E_alice, sigma', pi)`. The contract verifies π and the PS pairing relation via the BN254 precompiles (~235K gas, paid once for the lifetime of this Ethereum address). After this call, `isVerified[alice] = true` and `issuerOf[alice] = issuer`.

Alice repeats steps 2-3 for every Ethereum account she controls; the issuer never sees those accounts.

8.2 Step 1: Mint

Alice calls `BUCK.mint(50000e18)`:

1. The contract first checks `identity.isVerified(alice) = true`.
2. `buckCredit.totalCurrentValue(alice)` returns 341,000e18.
3. `buckK.currentBuckK()` returns the current stabilization factor. **Phase 0:** this is just the static value the governance multisig last wrote. **Eventual PID build:** the controller fetches oracles – e.g. commodity basket at 1.02 USDC, BUCK at 0.99 USDC, error +0.03 -> `BUCK_K = 1.015`.
4. `maxLimit = 341,000 * BUCK_K`. Assume `BUCK_K = 1.015`, so `maxLimit = 346,115`.
5. Alice's current balance is 280,000 BUCKs. After mint: `330,000 < 346,115`. OK.
6. Utilization after mint: `330,000 / 346,115 = 95.3%`.
 - Premium rate: $0.50\% + (0.953^2 * 4.50\%) = 4.59\%$.
 - Premium: `50,000 * 4.59% = 2,295` BUCKs to insurance pool.
7. Alice receives 47,705 BUCKs. Insurance pool receives 2,295 BUCKs.

8.3 Step 2: Approve Bob (Identity Handshake)

Alice wants to pay Bob 10,000 BUCKs through the same router pattern most ERC-20 flows use (`approve` + `transferFrom`). Bob is also registered.

1. Alice's wallet reads Bob's `pk_bob` from `IdentityRegistry`.
2. Off-chain, Alice's wallet decrypts `E_alice` (it knows `sk_alice`), recovers M , and re-encrypts under pk_{bob} with fresh randomness r_b : $E_{bob} = (r_b \cdot G, r_b \cdot pk_{bob} + M)$.
3. The wallet produces a Chaum-Pedersen NIZK proof $\pi_{CP} = (e, s_1, s_2)$ that `E_bob` encrypts the same M registered in `E_alice`. The Fiat-Shamir challenge binds `msg.sender = alice`, `spender = bob`, and `block.chainid`.
4. Alice calls `BUCK.approve(bob, 10000e18, E_bob, pi_CP)`. The contract:
 - checks both parties are verified;
 - asks `IdentityRegistry.verifyApprove(alice, bob, E_bob, pi_CP)` – the registry reads `E_alice` from its own storage (so Alice can't substitute a fake credential) and runs the CP check (~29K gas);

- stores `_receiptFragments[alice][bob] = keccak256(E_bob);`
 - records the standard ERC-20 allowance.
5. Off-chain, Alice’s Holochain hApp delivers the matching `identity_data` and E_{bob} ciphertext to Bob’s hApp. Bob’s wallet decrypts to recover M' and asserts $H(identity_data) \cdot G \stackrel{?}{=} M'$ – the two-channel binding from the Identity document.

8.4 Step 3: Bob Pulls the Transfer

Bob (or a contract Bob authorized) calls `BUCK.transferFrom(alice, bob, 10000e18):`

1. `_spendAllowance(alice, bob, 10000e18)` debits the allowance Alice set above.
2. `_identityCheckedTransfer(alice, bob, 10000e18)` verifies both parties are still verified and that `_receiptFragments[alice][bob] != 0` (Alice did supply a CP-bound receipt at approve time).
3. Tokens transfer.
4. `BuckTransferReceipt(alice, bob, 10000e18, fromHash, toHash)` is emitted, with `fromHash` and `toHash` being keccak256 commitments over the corresponding ciphertexts. An auditor with subpoena power over either Alice or Bob can recover the underlying identities; an outside observer learns only that addresses `alice` and `bob` exchanged 10,000 BUCKs.

8.5 Step 4: Alice Denominates 125 BUCK as Notes

Separately, Alice wants to move some of her BUCKs off the public-ledger transfer path into private notes. Her wallet picks flavour A2 and values [100 BUCK, 25 BUCK] (totalFace = 125 BUCK), draws fresh `rho[i]` and `idHash[i]` randomness, and – since the deployment pins $N \setminus \{16, 128, 1024\}$ – pads the live pair with 14 dummy openings whose $v_i = 0$ so the batch lands on the smallest pinned circuit ($N^* = 16$). The wallet snapshots (`oldRoot`, `nextLeafIndex`) from chain, derives the depth-20 sibling paths for each of the 16 leaves *in submission order*, and runs `snarkjs groth16 fullProve` against `build/snark/mint_batch_N16/mint.wasm` and `mint_final.zkey`. The proof triple (`pA`, `pB`, `pC`) is abi-encoded as (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) into `proofBytes`.

On-chain, Alice makes two calls:

1. `BUCK.approve(notes, 125e18, E_notes, pi_CP_notes)` – standard identity-aware approve. The `Notes` contract was registered `setSystemPublic(true)` by governance, so this branch takes the public-claim path (no CP proof is actually checked against Notes, though the ERC-20 allowance is still written).
2. `Notes.mint(proofBytes, oldRoot, newRoot, nextLeafIndex, 125e18, [cm[0], ..., cm[15]])` – the `MintVerifierAdapter` dispatches by `cms.length=16` to `mintVerifiers[16]` and forwards to the $N=16$ Groth16 verifier with public signals [`oldRoot`, `newRoot`, `nextLeafIndex`, `125e18`, `keccak256(cms)`]. On success, 125 BUCK moves from Alice to Notes, `roots[]` advances to the SNARK-attested `newRoot`, `nextLeafIndex` advances by 16, and `noteFaceSum + 125e18 =`. Alice’s wallet keeps the live openings (`flavor`, `v`, `rho`, `idHash`, `predicate` for the two real notes) privately – those are the `NoteOpening` records she will need to spend. The 14 dummy openings are discarded.

8.6 Step 5: Alice (or her recipient) Spends a Note

Later, the holder of one of Alice’s notes – it could be Alice herself, or the bearer to whom she off-chain handed the opening – redeems `cm[0]` for 100 BUCK to address `recipient`:

1. The wallet reconstructs the Merkle path (`pathElements[20]`, `pathIndices[20]`) for `cm[0]` by replaying the `Minted` event log and pulling the calldata `cms[]` out of each batch mint transaction (or from a locally-cached tree).
2. `snarkjs groth16 fullProve` against `build/snark/spend/spend.wasm` and `spend_final.zkey` produces a proof binding `[noteRoot, nullifier, face, recipient, chainId] = [currentRoot, Poseidon(rho, idHash, 4242), 100e18, recipient, 1]` to the witness opening.
3. `Notes.spend(proofBytes, currentRoot, nullifier, 100e18, recipient)` – the contract checks the root is still in the recent-roots window, rejects a previously-seen nullifier, runs the adapter (~360K gas total), marks the nullifier burned, decrements `noteFaceSum`, and calls `BUCK.transferCarrying(recipient, 100e18)`. The recipient receives the full 100 BUCK and an amount-weighted share of the pool’s average demurrage age.

Later, `HomeCo` reappraises the structure at \$280K (updated `faceValue`). Alice’s credit limit drops. If her outstanding balance exceeds the new limit, the default insurance mechanism activates (or the Jubilee fund eventually releases the lien), and she can no longer mint BUCKs.

9 Gas Cost Analysis

Approximate gas costs (Ethereum mainnet, Solidity 0.8.x optimizer). The Notes rows are measured from the `MintVerifier.t.sol` and `SpendVerifier.t.sol` Foundry runs against the real Groth16 verifiers; the rest are architectural estimates for the eventual PID build.

Operation	Gas (approx)	Cost at 30 gwei	Notes
<code>BUCK_CREDIT.activate()</code>	~50K	~\$4	Single SSTORE + event
<code>BUCK_CREDIT.updateCredit()</code>	~80K	~\$6	Multiple SSTOREs (insurer pays)
<code>IdentityRegistry.register()</code>	~235K	~\$19	PS pairing + NIZK (one-time)
<code>IdentityRegistry.verifyApprove</code>	~29K	~\$2	CP proof, per counterparty pair
<code>BUCK.approve()</code> (identity)	~97K	~\$8	29K CP + 46K allowance + 22K SSTORE
<code>BUCK.transfer</code> / <code>transferFrom</code>	~80K	~\$6	ERC-20 baseline + receipt event
<code>BUCK.transferCarrying()</code>	~60K	~\$5	Weighted-merge index, no fee burn
<code>BUCK.mint()</code> (static <code>BUCK_K</code>)	~150K	~\$12	Enumerate credits + mint
<code>BUCK.mint()</code> (with PID update)	~350K	~\$28	+ Oracle reads + PID computation
<code>Notes.mint()</code> (Groth16, N=2)	~1.9M	~\$152	Groth16 verify + 20-deep tree insert
<code>Notes.spend()</code> (Groth16)	~360K	~\$29	Groth16 verify + <code>transferCarrying</code>
<code>BUCK_CREDIT.currentValue()</code>	0 (view)	Free	Pure computation, no state change
<code>BUCK_K.currentBuckK()</code>	0 (view)	Free	Read cached value

The PID update cost (~200K gas additional) is amortized across all mints within a `dT` window. With a 1-hour `dT`, only the first mint after each hour pays for the oracle reads and PID computation.

For lower gas costs, the same contracts can be deployed on Ethereum L2s (Arbitrum, Optimism, Base) where gas is 10-100x cheaper, or on alternative EVM chains. The contract architecture is chain-agnostic.

10 Security Considerations

10.1 Oracle Manipulation

- **TWAP resistance:** The BUCK/USDC price uses a Uniswap V3 TWAP (30+ minutes), requiring sustained capital commitment to manipulate.
- **Chainlink feeds:** Commodity prices from Chainlink’s decentralized oracle network, with built-in deviation thresholds and heartbeat checks.
- **Staleness checks:** `_getBasketCost()` should verify `updatedAt` is within an acceptable window (e.g., 1 hour) and revert if stale.

10.2 Credit Enumeration DoS

A malicious user could create many tiny BUCK_CREDITS to make `totalCurrentValue()` gas-prohibitive. Mitigations:

- Minimum face value for BUCK_CREDIT creation.
- Cap on credits per account (e.g., 20).
- Off-chain aggregation with Merkle proof (advanced).

10.3 Insurer Trust

The insurer can reduce `faceValue` at any time, potentially pushing accounts into default. This is by design – it reflects real-world reappraisal risk. Mitigations:

- Insurance pool covers default events.
- Multiple competing insurers provide market discipline.
- Governance can blacklist abusive insurers.
- Time-locks on insurer updates (e.g., 7-day delay for reductions $> 20\%$).

10.4 Reentrancy

Standard OpenZeppelin `ReentrancyGuard` on `mint()` and `activate()`. All state changes occur before external calls.

11 Future Approaches

This ERC-721 implementation prioritizes clarity and correctness. Subsequent documents will explore:

11.1 ERC-1155: Semi-Fungible Batch Credits

If standardized credit classes emerge (e.g., "Calgary residential, HomeCo, LINEAR 200bp"), multiple properties in the same class could share a token ID with individual amounts. Benefits:

- Batch operations (`safeBatchTransferFrom`) for portfolio management.
- Reduced gas for accounts with many same-class credits.
- Natural fit for insurance tranches within a single credit.

Challenges: per-token depreciation start dates require additional state mapping, partially negating the fungibility benefit.

11.2 ERC-3643 (T-REX): Regulated Security Token

For production deployment under regulatory oversight:

- **ONCHAINID**: Native identity registry for KYC/AML compliance.
- **Compliance modules**: Programmable transfer restrictions (e.g., credits can only transfer to verified Alberta residents).
- **Agent roles**: Insurer as agent with granular permissions (update, freeze, force-transfer).
- **Recovery**: Lost-key recovery through identity verification.

This is the likely production standard if Alberta implements BUCK under provincial regulation.

11.3 ERC-4626: Vault-Based Insurance Pool

While a poor fit for BUCK_CREDIT itself, ERC-4626 is well-suited for the `InsurancePool`:

- Premium deposits as `deposit()` / mutual insurance shares as `shares`.
- `convertToAssets()` / `convertToShares()` for transparent share pricing.
- Standard DeFi composability (yield aggregators, lending protocols).

11.4 L2 and Cross-Chain Deployment

The contracts are EVM-compatible and can deploy on Arbitrum, Optimism, Base, or Polygon with minimal changes. Cross-chain BUCK transfers via canonical bridges or LayerZero/Axelar would enable multi-chain liquidity while maintaining a single BUCK_K oracle on the settlement layer.

12 Implementation Plan

This document specifies the **eventual** shape of the system. The first deployment ships in narrow phases so the cryptographic identity layer – the single largest piece of new ground in the design – can land first, against minimal stubs of the surrounding contracts. Each phase ends in a self-contained, deployable state with passing tests. The status table below reflects the repo at the header date; the phase subsections document the intent of each phase as originally scoped and are annotated with what the current code actually ships.

12.1 Phase Completion Status (current)

Phase	Scope	Status
Phase 0	Stub <code>BuckCredit</code> + <code>BuckKControllerStatic</code>	done – <code>BuckKControllerStatic.sol</code> live
Phase 1	<code>BN254.sol</code> helper library	done – <code>ecAdd/ecMul/ecPairing</code> wrappers + FS helper
Phase 2	<code>IdentityRegistry.sol</code>	done – PS + NIZK + CP all on-chain
Phase 3	Identity-aware <code>Buck.sol</code>	done – CP-bound approve, bilateral transfer, receipt
Phase 4	Public-claim accounts	done – <code>setPublic</code> / <code>setSystemPublic</code> ; Notes wired a
Phase 5	Notes Phase-1 scaffolding	done – <code>Notes.sol</code> + <code>StubMintVerifier</code> ; commitmen
Phase 6	Groth16 mint circuit + adapter	done – <code>circom mint.circom</code> (N=2), <code>MintGroth16Ver</code>
Phase 7	Notes Phase-3 spend circuit + nullifier reveal	done – <code>spend.circom</code> , <code>SpendGroth16Verifier</code> , Torna
Phase 8	Live <code>BuckKController</code> (PID + oracle)	pending – contract skeleton exists; oracle hookup and
Phase 9	Production MPC trusted-setup ceremony	pending – <code>scripts/snark/setup.sh</code> uses dev-only e

Current forge suite: 100 local-EVM unit tests pass; one `BuckKControllerForkTest` requires a live mainnet RPC and is skipped offline.

12.2 Testing Strategy: Three Layers

Tests are stratified by what they actually need from the EVM, so the cryptographic and accounting work can iterate without ever touching a network:

1. **Local-EVM unit tests (default).** Foundry’s in-memory EVM ships the BN254 precompiles (`ecAdd 0x06`, `ecMul 0x07`, `ecPairing 0x08`) and all the EIP-2929 / cancun behavior we need. Every contract in this document – `BUCK_CREDIT`, `BUCK`, `IdentityRegistry`, the BN254 helper, the static `BUCK_K` – is fully exercisable without RPC, on a clean state, in milliseconds per test.
2. **Pinned mainnet/Sepolia forks (only the eventual PID `BUCK_K`).** When the production `BuckKController` ships and needs to read Chainlink price feeds and the Uniswap V3 `BUCK/USDC TWAP`, run the relevant tests with `--fork-url ${RPC} --fork-block-number ${N}`. Pinning the block number is the determinism knob: the EVM state at block N is immutable, and Foundry caches the RPC responses under `~/.foundry/cache/rpc/` – after the first run the suite is offline and bit-reproducible. `vm.mockCall` covers pathological oracle paths (stale feeds, manipulated TWAPs) without needing to find a block at which they actually occurred.
3. **Differential parity with the Python reference.** The cryptographic surface is described by an executable Python reference (see **Off-Chain Wallet** below). CI regenerates the canonical JSON test vectors from that reference and `git diff --exit-code`s them against committed copies; the Solidity tests load the same vectors via `vm.readFile` and assert equality. Any drift between the spec, the wallet, or the Solidity tells us *immediately* which of the three is wrong.

The deliberate exclusion: no `vm.ffi` shelling out to Python at test time. That trades determinism, speed, and CI portability for a coupling we don’t need – the wallet emits vectors once, the Solidity reads them many times.

12.3 Off-Chain Wallet (`alberta_buck.wallet`, Python)

The off-chain components – KYC ceremony, credential derivation, Chaum-Pedersen proving, and (later) SNARK witness generation for Notes – are implemented as a Python package, `alberta_buck.wallet`, with a CLI front-end. Python is the natural choice: the existing references in Identity Example and Proofs are already Python, `py_ecc` ships a complete BN254 (alt_bn128) implementation matching Ethereum’s precompiles, and the package is invoked the same way every other dev tool in this repo is (`nix develop --command python3 -m alberta_buck.wallet ...`).

The wallet has three jobs:

- **Reference implementation** of the cryptography in Identity, the place where the spec is ground-truthed.
- **Test-vector emission:** a CLI verb (`wallet emit-vectors`) writes a canonical JSON file under `test/vectors/` containing the inputs and expected outputs of every on-chain proof check. Foundry tests load this file and assert agreement with the Solidity verifier.
- **End-to-end developer surface:** a CLI verb (`wallet sign`, `wallet register`, `wallet approve`) produces the calldata an end user’s wallet would submit, so integration tests (and humans) can drive the contracts the same way a real wallet would.

The wallet does **not** hold private keys for production users; it is a deterministic reference and a test driver. A production wallet would reuse the same primitives behind a different storage and UI.

12.4 Phase 0: Stub the Surrounding Contracts

Goal: Buck can be developed and tested against a minimal, deterministic surface for `BuckCredit` and `BuckK`. All real moving parts (depreciation curves, PID, oracles) are deferred.

- `BuckKController.sol` -> replace with `BuckKControllerStatic.sol`:
 - State: `uint256 public buckK` (18 decimals, default `1e18`).
 - Functions: `currentBuckK()` `view` returns `(uint256)`, `compute()` (no-op compat shim), `setBuckK(uint256)`, `transferGovernance(address)` (both governance-only).
 - Drop all PID state, Chainlink basket, and Uniswap TWAP code. The full PID implementation documented above remains the design target – it slots back in once the identity layer is stable. The design-target `BuckKController.sol` also ships (unit-tested with mocked feeds); the live-oracle fork test is gated on RPC availability.
- `BuckCredit.sol` -> keep the existing implementation, but treat only one function as a load-bearing interface for Phase 0+: `totalCurrentValue(address holder)` `external view` returns `(uint256)`. No depreciation rework, no insurer-flow churn.

Deliverables: passing Foundry tests that mint, set new `BUCK_K` values, and watch credit limits re-derive correctly.

12.5 Phase 1: BN254 Helper Library

Goal: a single Solidity library that wraps the three BN254 precompiles and exposes the typed operations the identity contracts need.

- `src/crypto/BN254.sol` with:
 - `G1Point`, `G2Point` structs and constants for `g1`, `g2`.
 - `add(G1Point, G1Point) -> G1Point` via `ecAdd` precompile (0x06).
 - `mul(G1Point, uint256) -> G1Point` via `ecMul` precompile (0x07).
 - `pairingProd3(...)` and `pairingProd4(...)` helpers via `ecPairing` precompile (0x08).
 - Pure-Solidity Fiat-Shamir helpers: `fsRegistration(...)` and `fsChaumPedersen(...)`, each wrapping a `keccak256` over the canonical encoding from the Identity document.
 - High-level checks: `checkPSSignature`, `checkRegistrationNIZK`, `checkChaumPedersen`.

Deliverables: differential tests against the Python reference in `alberta-buck-identity-example.org` (same curve, same encoding, same Fiat-Shamir transcript). `nix develop --command python3` drives the reference; Foundry `vm.ffi` or pre-baked vector files feed the Solidity tests.

12.6 Phase 2: IdentityRegistry Contract

Goal: ship the on-chain trust anchor exactly as specified in the IdentityRegistry section.

- `src/IdentityRegistry.sol` implementing `register`, `setPublic`, `verifyApprove`, `trustIssuer`.
- All proofs verified using BN254 helpers from Phase 1.
- Tests:
 - End-to-end registration with the Python reference vectors (positive case + the four counter-examples from `alberta-buck-identity-example.org`).
 - `register` rejects untrusted issuers.
 - `register` rejects double-registration of the same address.
 - `verifyApprove` accepts a valid Chaum-Pedersen proof and rejects every documented tampered case (wrong message, wrong randomness, wrong public key, replayed challenge).
 - Gas measurements within ~15% of the published estimates (235K register / 29K verifyApprove).

Deliverables: deployable registry, passing test suite, documented gas measurements.

12.7 Phase 3: Identity-Aware Buck ERC-20

Goal: replace the current `Buck.sol` with the identity-aware version specified in the BUCK section.

- Add `IIdentityRegistry` interface and constructor wiring.
- mint gates on `isVerified`.
- `approve(spender, amount, E_bob, pi_CP)` with the parameterless `approve(address, uint256)` reverting.
- `transfer` / `transferFrom` via `_identityCheckedTransfer` with bilateral checks and the `BuckTransferReceipt` event.

- `_receiptFragments` storage and `_publicReceiptHash` helper.

Tests:

- Two registered users can complete the full `approve -> transferFrom` cycle and observe a `BuckTransferReceipt` whose hashes match the off-chain ciphertexts.
- Unregistered `mint` reverts.
- `transfer` to an unregistered, non-public recipient reverts.
- `transfer` without a prior CP-bound `approve` (and a non-public recipient) reverts.
- Public-claim recipients accept transfers without an `approve` step.

12.8 Phase 4: Public-Claim Account Support

Goal: AMM pools, vaults, and treasuries can interact with BUCK without needing to generate Chaum-Pedersen proofs.

- Confirm `setPublic(true)` is invocable only by registered accounts.
- Add a small `MockAMMPool` test contract that `register=s`, `=setPublic(true)=s`, then participates in `=approve=/=transferFrom` flows on both sides (taker and maker).
- Verify the receipt event uses the well-known `M` for the public side and the CP-bound ciphertext for the private side.

12.9 Phase 5: Notes Scaffolding

Goal: lay the structural groundwork for BUCK Notes – a commitment pool, a pluggable mint verifier interface, and the off-chain wallet helpers – behind a stub verifier so the end-to-end flow can be exercised without yet shipping the SNARK.

Shipped:

- `src/Notes.sol` – the contract documented in the Notes section above. The initial Phase-5 deployment used a flat `commitments[]` append list behind a stub verifier; the incremental Poseidon Merkle tree and 30-slot recent-roots ring buffer landed alongside Phase 7.
- `src/IMintVerifier.sol` / `src/ISpendVerifier.sol` – the narrow verifier interfaces. Decoupling Notes from the SNARK toolchain was the key Phase-5 invariant so the Phase-6 / Phase-7 verifiers land as constructor-argument swaps.
- `src/StubMintVerifier.sol` / `src/StubSpendVerifier.sol` – governance-toggable boolean verifiers used by all Notes state-machine tests.
- `alberta_buck.wallet.notes` (Python) – `NoteOpening` dataclass, flavour constants, keccak-based placeholder commitment / nullifier helpers with domain-separating tags, `id_payload_a1/a2/b1` helpers matching the expected circuit public-input layout. 13 pytest tests.
- `test/Notes.t.sol` – 16 tests including happy path, duplicate detection, zero-commitment rejection, empty batch, verifier disabled, external rejecting verifier, missing approval, missing balance, state-atomicity on transfer failure.

The Python wallet's `note_commitment` is a keccak-based placeholder; the on-chain contract stores opaque `uint256` commitments without re-hashing, so swapping keccak for Poseidon is an off-chain wallet change only.

12.10 Phase 6: Groth16 Mint Circuit + Verifier

Goal: turn the stub into a cryptographically load-bearing verifier. The mint contract must refuse to append a batch unless a real Groth16 proof ties `totalFace` to the sum of the (flavour, value, rho, idHash, predicate) openings, and each `cm[i]` to `Poseidon(5)` of its opening.

Shipped:

- `flake.nix` adds `circom` to the dev shell and bootstraps npm dev dependencies on first entry.
- `package.json` pins `circomlib` 2.0.5, `circomlibjs` ^0.1.7, `snarkjs` 0.7.5, `ethers` ^6.13.4.
- `circuits/mint.circom` – $N = 2$ mint circuit (see Notes section for the signal table), with `Num2Bits(128)` range bounds on `totalFace` and each `v[i]`. ~3,500 R1CS constraints.
- `scripts/snark/setup.sh` – `circom` compile, 2^{15} powers-of-tau (shared between mint and spend), groth16 phase-2 setup, export verification key + Solidity verifier (`MintGroth16Verifier`, `SpendGroth16Verifier`). Dev-only entropy.
- `scripts/snark/prove_mint.js` – witness builder (two A2 notes, `v[100,25]e18=`), seeded LCG randomness, `snarkjs.groth16.fullProve`, pB Fp2-coordinate swap, abi-encoded `proofBytes`, and sanity-check `snarkjs.groth16.verify`.
- `src/MintGroth16Verifier.sol` – snarkjs-generated verifier.
- `src/MintVerifierAdapter.sol` – `MINT_BATCH_SIZE=2` adapter implementing `IMintVerifier`.
- `test/MintVerifier.t.sol` – 7 tests: happy path (real proof accepted), three tamper attacks (`totalFace`, `cm[0]`, batch size) all reverting with `Notes: bad mint proof`, malformed proof-bytes revert at decode, adapter constructor rejects zero.

The circuit is pinned at $N=2$. A variable- N deployment re-templates both the circuit and the adapter's public-signals arity.

12.11 Phase 7: Spend Circuit + Nullifier Reveal

Goal: cryptographically redeem a minted commitment against an on-chain Merkle root under a fresh nullifier, paying out an equal face value of BUCK via `transferCarrying` so the SNARK's `face == v` invariant is preserved at the ERC-20 boundary.

Shipped:

- `circuits/spend.circom` – single-note B-shape redemption (~12,000 R1CS constraints). Public inputs [`noteRoot`, `nullifier`, `face`, `recipient`, `chainId`]; five constraint groups (range, commitment, Merkle inclusion at depth 20, nullifier determinism `Poseidon(3)` (`rho`, `idHash`, 4242), value conservation). A-flavor in-circuit identity binding is deferred to `spend_a.circom` (a follow-up phase).
- `src/SpendGroth16Verifier.sol` and `src/SpendVerifierAdapter.sol` – snarkjs-generated verifier and the narrow `ISpendVerifier` adapter.

- `src/Notes.sol` extended with the Tornado-style incremental Poseidon Merkle tree (depth 20, `TREE_DEPTH` precomputed empty-subtree roots, 30-slot `roots[]` ring buffer), and the new `spend(proof, root, nullifier, face, recipient)` entry point.
- `src/PoseidonT3Bytecode.sol` – a helper that deploys the 2-input Poseidon precompile from raw bytecode generated by `circomlibjs`; `script/Deploy.s.sol` deploys it and passes the address to `Notes`’s constructor.
- `src/Buck.sol` extended with continuous demurrage (`cumIndex`, `BASE_RATE_PER_SEC`, per-account `_indexAtLastTouch`) and `transferCarrying(to, amount)` – a single entry point that performs a weighted-index merge on the recipient without burning a fee, so a note redemption pays the recipient `face`, not `face - fee`. (The per-account `FeeTreatment` enum that earlier drafts proposed has been replaced by this simpler single-function approach; the `Notes` pool does not need a flag because `transferCarrying` is the only path it calls.)
- `test/SpendVerifier.t.sol` and `test/BuckDemurrage.t.sol` cover: happy path, double-spend, unknown root, tampered recipient/face/nullifier, chainId mismatch, zero-recipient, and the weighted-merge invariant that the recipient’s age equals the pool’s average after spend.

Pending:

- **Phase 8:** live `BuckKController` deployment. The contract is already in-tree with mock-feed unit tests; the remaining work is oracle wiring (Chainlink feed addresses, Uniswap V3 pool), governance gain-setting, and a live mainnet fork test cycle before flipping the `Buck` constructor to point at it instead of the `Static` variant.
- **Phase 9:** production MPC ceremony to replace the dev-only `scripts/snark/setup.sh` trusted-setup entropy.
- **A-flavor spend:** `spend_a.circom` for redemptions that carry an in-circuit CP-equality / ElGamal identity binding. Orthogonal to Phase 8/9; shares the existing Merkle-tree and nullifier infrastructure.

12.11.1 Resolved Items from the Notes Correctness Review

The Phase 6 → Phase 7 transition surfaced four correctness obligations originally listed as open questions in `alberta-buck-proofs.org` (Part IV). All four are now resolved in the shipped contracts:

1. **Num2Bits range checks on `v[i]` and `totalFace` in `circuits/mint.circom` – *resolved*.** `vRange[i] = Num2Bits(128)` and `totalRange = Num2Bits(128)` are wired in `mint.circom`; `scripts/snark/setup.sh` and `MintGroth16Verifier` have been refreshed. The spend circuit re-derives the matching range bound on its own `v` (`vRange = Num2Bits(128)`) and on the public `face` (`faceRange = Num2Bits(128)`), closing the field-overflow corridor end-to-end.
2. **Explicit batch identifier in `Notes.mint` – *still an open design question, not a correctness bug*.** The shipped `Minted(issuer, totalFace, startIndex, count, newRoot)` event carries a root per batch, which an indexer can Poseidon-commit to if an auditable `batchId` is wanted. Revisit if regulatory analytics need an on-chain `batchIds` map.
3. **Phase-7 spend circuit and on-chain support – *resolved*.** Shipped as described above: the Tornado-style incremental Poseidon tree lives in `Notes.sol`, nullifiers are computed in-circuit as `Poseidon(3)(rho, idHash, 4242)`, and value conservation is enforced via the 128-bit range bound plus the commitment reconstruction constraint.

4. **Per-account fee-treatment mode** – *superseded*. The earlier draft added an enum `FeeTreatment` { `Deducting`, `Carrying` }, a per-account `_feeMode` mapping, a `BuckDerived()` bytecode marker, a `balanceOf(account) = 0` precondition for mode changes, and a `FeeCarried` event. The shipped solution is simpler: a single `transferCarrying(to, amount)` method on `Buck.sol` that performs the weighted-merge index update atomically, callable by any contract (today only `Notes` calls it; future escrow/payroll contracts may too). No per-account mode, no bytecode marker, and no balance precondition are needed because the weighted merge is continuous – the recipient simply absorbs a proportional share of the sender’s unsettled age.

12.12 Sequencing and Stop Conditions

The phases are strict prerequisites of one another: Phase $n+1$ does not begin until Phase n has a passing CI run and a Foundry gas snapshot. Implementation **STOP** conditions:

- Phase 1: a Solidity `checkChaumPedersen` disagreement with the Python reference vectors. Disagreement implies a bug in the spec or in one of the implementations – resolve before any contract that depends on it ships.
- Phase 3: `BuckTransferReceipt` hash collisions across distinct ciphertexts (would indicate an ABI-encoding bug – canonicalisation matters).
- Phase 5: any Solidity-side type that does not match the chosen SNARK verifier output (revisit the toolchain choice; do not paper over with adapter contracts).
- Phase 6: a tamper-path test that fails to revert. The adapter + Groth16 verifier must reject every documented attack on `totalFace`, `cm[i]`, batch size, and malformed proof bytes; any false-accept is a hard stop before further `Notes` work.
- Phase 7: any inconsistency between the mint-circuit commitment layout and the spend circuit’s opening layout (would fork the two proof systems’ views of a note, silently). Both circuits are pinned at `Poseidon(5)(flavor, v, rho, idHash, predicate)` with `Num2Bits(128)` range bounds on every `v[i]`, `totalFace`, and `face`; the value-conservation constraint `sum(v[i]) == totalFace` now holds in the integers (62-bit margin against field overflow at $N < 2^{64}$). Any future change to either circuit’s commitment layout must be made simultaneously in both.